ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC & KỸ THUẬT MÁY TÍNH



HỆ THỐNG NHÚNG (CO3054)

BÁO CÁO BÀI THỰC HÀNH 2

CƠ CHẾ ĐỒNG BỘ TRONG RTOS

Lớp: L03 - Học Kỳ HK261

Ngày nộp: 12/11/2026

GVHD: Thầy Lê Phương Nam

Sinh Viên	MSSV
Phạm Công Võ	2313946

TP.Hồ Chí Minh, Tháng 11/2026

THÔNG TIN THỰC HIỆN VÀ ĐÁNH GIÁ KẾT QUẢ

BÀI THỰC HÀNH 2

STT	Họ và tên	MSSV	Nhiệm vụ	Kết quả	Chữ ký
1	Phạm Công Võ	2313946	• Hiện thực toàn bộ mã nguồn Bài 1 đến Bài 5 • Thực nghiệm trên Kit BKIT ARM4 • Soạn thảo tài liệu báo cáo	100%	

Mục lục

1 Tổng Quan và Mục Tiêu Bài Thực Hành 2 **3**
1.1 Giới thiệu . . . 3
1.2 Mục tiêu bài thực hành . . . 3
1.3 Quy định kỹ thuật bắt buộc . . . 3

2 Nội Dung Chi Tiết Các Bài Tập Thực Hành **4**
2.1 Bài 1: Chạy tiếp sức ba đèn bằng Binary Semaphore . . . 4
2.1.1 Mã nguồn chương trình (`src/main.c`) . . . 4
2.1.2 Kết quả thực nghiệm & Log Serial Monitor . . . 5
2.1.3 Trả lời câu hỏi bài tập . . . 6
2.2 Bài 2: Giám sát tín hiệu có Timeout . . . 7
2.2.1 Mã nguồn chương trình (`src/main.c`) . . . 7
2.2.2 Kết quả thực nghiệm & Log Serial Monitor . . . 8
2.2.3 Trả lời câu hỏi bài tập . . . 9
2.3 Bài 3: Chia sẻ 2 kênh output cho 4 worker bằng Counting Semaphore . . . 10
2.3.1 Mã nguồn chương trình (`src/main.c`) . . . 10
2.3.2 Kết quả thực nghiệm & Log Serial Monitor . . . 12
2.3.3 Trả lời câu hỏi bài tập . . . 13
2.4 Bài 4: Thu thập dữ liệu từ ISR qua Message Queue . . . 14
2.4.1 Mã nguồn chương trình (`src/main.c`) . . . 14
2.4.2 Kết quả thực nghiệm & Log Serial Monitor . . . 16
2.4.3 Trả lời câu hỏi bài tập . . . 17
2.5 Bài 5: Đèn giao thông có nút xin qua đường (Bài tổng hợp) . . . 18
2.5.1 Mã nguồn chương trình (`src/main.c`) . . . 18
2.5.2 Kết quả thực nghiệm & Log Serial Monitor . . . 21
2.5.3 Trả lời câu hỏi bài tập . . . 22

Tài liệu tham khảo **24**

1 Tổng Quan và Mục Tiêu Bài Thực Hành 2

1.1 Giới thiệu

Bài thực hành 2 tập trung vào các cơ chế đồng bộ và giao tiếp giữa các tiến trình (Inter-Process Communication - IPC) trong hệ điều hành thời gian thực Zephyr RTOS trên Kit thí nghiệm BKIT ARM4 (STM32F407ZGT6). Nội dung khảo sát việc ứng dụng Binary Semaphore, Counting Semaphore, Mutex và Message Queue nhằm giải quyết triệt để các bài toán kinh điển trong lập trình hệ thống nhúng: điều phối thứ tự thực thi tuần tự, loại trừ lẫn nhau bảo vệ vùng găng, và truyền nhận dữ liệu an toàn phi phong tỏa giữa ngắt phần cứng (ISR) với luồng thực thi (Thread).

1.2 Mục tiêu bài thực hành

- Vận dụng Binary Semaphore để thiết lập trật tự thực thi nghiêm ngặt giữa nhiều thread chạy song song.
- Sử dụng Semaphore có cơ chế Timeout để phát hiện mất tín hiệu định kỳ và xử lý an toàn lỗi ngắt.
- Kết hợp Counting Semaphore và Mutex để quản lý hồ tài nguyên dùng chung (Resource Pool) gồm 2 kênh phần cứng cho 4 tác vụ đồng thời.
- Thực hiện giao tiếp một chiều bất đồng bộ giữa ISR và Thread thông qua Message Queue, phân tích hiện tượng tràn bộ đệm khi mất cân bằng thông lượng.
- Thiết kế hệ thống đèn giao thông thông minh hoàn chỉnh: máy trạng thái FSM, ngắt nút bấm người đi bộ, và kiến trúc ghi log tập trung qua Message Queue.

1.3 Quy định kỹ thuật bắt buộc

- Tuyệt đối không chờ bằng vòng lặp rỗng hỏi vòng (`while (!flag) {}`) hay `k_busy_wait()`: mọi sự đồng bộ phải dùng Semaphore, Mutex hoặc Message Queue.
- Trong ngữ cảnh ngắt ISR: Chỉ sử dụng các API non-blocking với tham số `K_NO_WAIT`; tuyệt đối không gọi `k_msleep`, `k_mutex_lock`, hay `printf`.
- Khởi tạo tĩnh các đối tượng kernel bằng các macro chuẩn: `K_SEM_DEFINE`, `K_MUTEX_DEFINE`, và `K_MSGQ_DEFINE`.
- Mọi thread phải được khởi tạo bằng `K_THREAD_DEFINE` với đúng định danh và độ ưu tiên theo yêu cầu.

2 Nội Dung Chi Tiết Các Bài Tập Thực Hành

2.1 Bài 1: Chạy tiếp sức ba đèn bằng Binary Semaphore

Yêu cầu đề bài (Trích phiếu bài tập)

- **Viết ba thread `thread_led3`, `thread_y0`, `thread_y1`, cùng độ ưu tiên, mỗi thread chỉ điều khiển một đèn.**
- **Ba đèn sáng lần lượt, đúng thứ tự LED3 → Y0 → Y1 → LED3 → ..., mỗi đèn sáng 500 ms rồi tắt và "chuyển lượt"cho đèn kế tiếp. Không bao giờ có hai đèn cùng sáng.**
- **Việc chuyển lượt chỉ được dùng ba binary semaphore (`sem_led3`, `sem_y0`, `sem_y1`): mỗi thread take semaphore của mình, làm việc, rồi give semaphore của thread kế tiếp. Không dùng biến toàn cục, không dùng `k_msleep` để canh giờ cho thread khác.**
- **Mỗi lần đến lượt, thread in ra console: `[thời điểm] TÊN_ĐÈN`.**

2.1.1 Mã nguồn chương trình (`src/main.c`)

```c
#include <stdio.h>
#include <zephyr/kernel.h>
#include <zephyr/drivers/gpio.h>

/* Khai bao phan cung LED tu Devicetree */
#define LED0_NODE DT_ALIAS(led0) /* LED3 - PE4 */
#define LED1_NODE DT_ALIAS(led1) /* Y0   - PE5 */
#define LED2_NODE DT_ALIAS(led2) /* Y1   - PE6 */

static const struct gpio_dt_spec led3 = GPIO_DT_SPEC_GET(LED0_NODE, gpios);
static const struct gpio_dt_spec y0   = GPIO_DT_SPEC_GET(LED1_NODE, gpios);
static const struct gpio_dt_spec y1   = GPIO_DT_SPEC_GET(LED2_NODE, gpios);

/* Khoi tao 3 binary semaphore: sem_led3 chay truoc (= 1), hai sem con lai = 0 */
K_SEM_DEFINE(sem_led3, 1, 1);
K_SEM_DEFINE(sem_y0,   0, 1);
K_SEM_DEFINE(sem_y1,   0, 1);

#define BLINK_TIME_MS 500

/* Thread dieu khien LED3 */
void fn_thread_led3(void *p1, void *p2, void *p3)
{
    gpio_pin_configure_dt(&led3, GPIO_OUTPUT_INACTIVE);
    while (1) {
        k_sem_take(&sem_led3, K_FOREVER);
        printf("[%6u ms] LED3\n", k_uptime_get_32());
        gpio_pin_set_dt(&led3, 1);
        k_msleep(BLINK_TIME_MS);
        gpio_pin_set_dt(&led3, 0);
```

```
        k_sem_give(&sem_y0);
    }
}

/* Thread dieu khien Y0 */
void fn_thread_y0(void *p1, void *p2, void *p3)
{
    gpio_pin_configure_dt(&y0, GPIO_OUTPUT_INACTIVE);
    while (1) {
        k_sem_take(&sem_y0, K_FOREVER);
        printf("[%6u ms] Y0\n", k_uptime_get_32());
        gpio_pin_set_dt(&y0, 1);
        k_msleep(BLINK_TIME_MS);
        gpio_pin_set_dt(&y0, 0);
        k_sem_give(&sem_y1);
    }
}

/* Thread dieu khien Y1 */
void fn_thread_y1(void *p1, void *p2, void *p3)
{
    gpio_pin_configure_dt(&y1, GPIO_OUTPUT_INACTIVE);
    while (1) {
        k_sem_take(&sem_y1, K_FOREVER);
        printf("[%6u ms] Y1\n", k_uptime_get_32());
        gpio_pin_set_dt(&y1, 1);
        k_msleep(BLINK_TIME_MS);
        gpio_pin_set_dt(&y1, 0);
        k_sem_give(&sem_led3);
    }
}

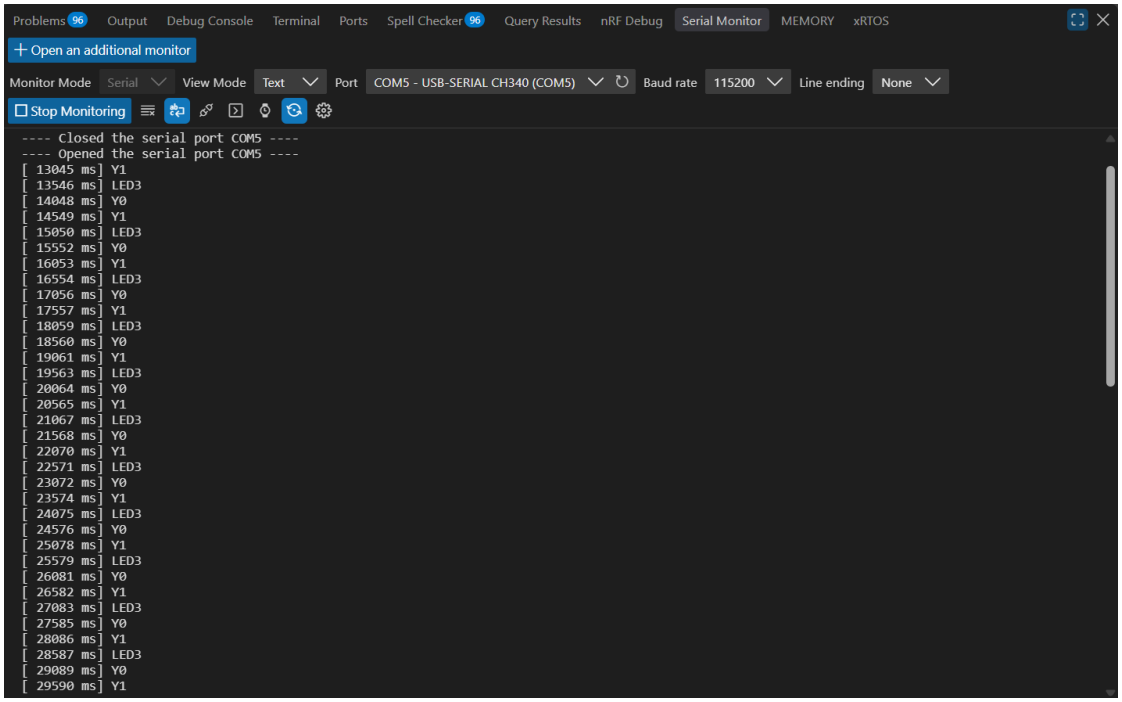
/* Dang ky 3 thread cung muc uu tien (Priority = 5) */
K_THREAD_DEFINE(thread_led3, 1024, fn_thread_led3, NULL, NULL, NULL, 5, 0, 0);
K_THREAD_DEFINE(thread_y0,   1024, fn_thread_y0,   NULL, NULL, NULL, 5, 0, 0);
K_THREAD_DEFINE(thread_y1,   1024, fn_thread_y1,   NULL, NULL, NULL, 5, 0, 0);

int main(void)
{
    printf("=== BKIT ARM4: Lab 2 - Bai 1: Tiep suc 3 den bang Binary Semaphore ===\n");
    return 0;
}
```

2.1.2 Kết quả thực nghiệm & Log Serial Monitor

Dưới đây là mẫu log tiêu biểu trích xuất từ Serial Monitor (115200 baud) và ảnh chụp màn hình thực nghiệm thực tế trên Kit BKIT ARM4:

```
=== BKIT ARM4: Lab 2 - Bai 1: Tiep suc 3 den bang Binary Semaphore ===
[     0 ms] LED3
[   500 ms] Y0
[  1000 ms] Y1
[  1500 ms] LED3
[  2000 ms] Y0
[  2500 ms] Y1
[  3000 ms] LED3
[  3500 ms] Y0
[  4000 ms] Y1
```



Hình 1: Minh chứng log Serial Monitor trên kit thật (115200 baud) - Bài 1

2.1.3 Trả lời câu hỏi bài tập

(a) **Bạn chọn giá trị ban đầu của ba semaphore như thế nào? Điều gì xảy ra nếu cả ba đều bằng 0? Nếu cả ba đều bằng 1?**
Trả lời:

- **Cấu hình chọn:** `sem_led3 = 1`, `sem_y0 = 0`, `sem_y1 = 0`. Việc gán `sem_led3 = 1` nhằm cấp quyền thực thi đầu tiên cho `thread_led3` ngay khi khởi động, trong khi hai luồng còn lại chuyển sang trạng thái blocked chờ lượt từ luồng trước đó.
- **Nếu cả ba đều bằng 0:** Xảy ra hiện tượng **Deadlock** (khóa chết hệ thống). Cả ba luồng khi khởi động đều gọi `k_sem_take` và lập tức bị chuyển sang trạng thái BLOCKED vĩnh viễn vì không có luồng nào được cấp vé ban đầu. Kết quả là hệ thống đứng im, không có đèn nào sáng.
- **Nếu cả ba đều bằng 1:** Cả ba luồng đều vượt qua lệnh `k_sem_take` ngay lập tức ở lần chạy đầu tiên. Khi đó cả ba đèn LED3, Y0, Y1 sẽ cùng bật sáng đồng thời trong 500 ms đầu, vi phạm hoàn toàn yêu cầu của đề bài là "không bao giờ có hai đèn cùng sáng".

(b) **Muốn đổi thứ tự thành LED3 → Y1 → Y0, bạn phải sửa những gì?**
Trả lời:

Ta chỉ cần sửa đối tượng semaphore được giải phóng trong lệnh `k_sem_give` ở cuối mỗi hàm luồng:

- Trong `fn_thread_led3`: Sửa `k_sem_give(&sem_y0)` thành `k_sem_give(&sem_y1)` (chuyền lượt cho Y1).
- Trong `fn_thread_y1`: Sửa `k_sem_give(&sem_led3)` thành `k_sem_give(&sem_y0)` (chuyền lượt cho Y0).
- Trong `fn_thread_y0`: Sửa `k_sem_give(&sem_y1)` thành `k_sem_give(&sem_led3)` (chuyền ngược về LED3 để khép kín chu trình).

Lưu ý: Giá trị khởi tạo ban đầu vẫn giữ: `sem_led3 = 1`, `sem_y1 = 0`, `sem_y0 = 0`.

2.2 Bài 2: Giám sát tín hiệu có Timeout

Yêu cầu đề bài (Trích phiếu bài tập)

- **Tạo một `k_timer` chạy chế độ one-shot; trong callback (ISR), sau khi `k_sem_give(&sig_sem)`, tự khởi động lại timer với chu kỳ lần lượt 1 s, 1 s, 4 s rồi lặp lại (mô phỏng tín hiệu thỉnh thoảng bị mất).**
- **Thread `thread_monitor` chờ tín hiệu bằng `k_sem_take(&sig_sem, K_MSEC(2000))` và kiểm tra giá trị trả về:**
  - **`0` → in `Nhan tin hieu`, nháy Y0 hai lần (mỗi lần 100 ms sáng / 100 ms tắt).**
  - **`-EAGAIN` → in `TIMEOUT - mat tin hieu!` và đảo trạng thái LED3.**
- **ISR không được in log, không điều khiển đèn — chỉ give và khởi động lại timer.**

2.2.1 Mã nguồn chương trình (`src/main.c`)

```c
#include <stdio.h>
#include <errno.h>
#include <zephyr/kernel.h>
#include <zephyr/drivers/gpio.h>

#define LED0_NODE DT_ALIAS(led0) /* LED3 - PE4: Den canh bao Timeout */
#define LED1_NODE DT_ALIAS(led1) /* Y0   - PE5: Den bao nhan tin hieu */

static const struct gpio_dt_spec led3 = GPIO_DT_SPEC_GET(LED0_NODE, gpios);
static const struct gpio_dt_spec y0   = GPIO_DT_SPEC_GET(LED1_NODE, gpios);

/* Khoi tao Binary Semaphore: ban dau = 0, toi da = 1 */
K_SEM_DEFINE(sig_sem, 0, 1);

static const int delay_pattern_ms[] = {1000, 1000, 4000};
static int pattern_idx = 0;

/* ISR cua Timer: Chi give semaphore va khoi dong lai timer, khong goi printf/sleep */
static void sig_timer_isr(struct k_timer *timer)
{
    k_sem_give(&sig_sem);
    k_timer_start(timer, K_MSEC(delay_pattern_ms[pattern_idx]), K_NO_WAIT);
    pattern_idx = (pattern_idx + 1) % 3;
}
K_TIMER_DEFINE(sig_timer, sig_timer_isr, NULL);

/* Thread giam sat tin hieu */
void fn_thread_monitor(void *p1, void *p2, void *p3)
{
    gpio_pin_configure_dt(&led3, GPIO_OUTPUT_INACTIVE);
    gpio_pin_configure_dt(&y0,   GPIO_OUTPUT_INACTIVE);

    while (1) {
        int ret = k_sem_take(&sig_sem, K_MSEC(2000));

```

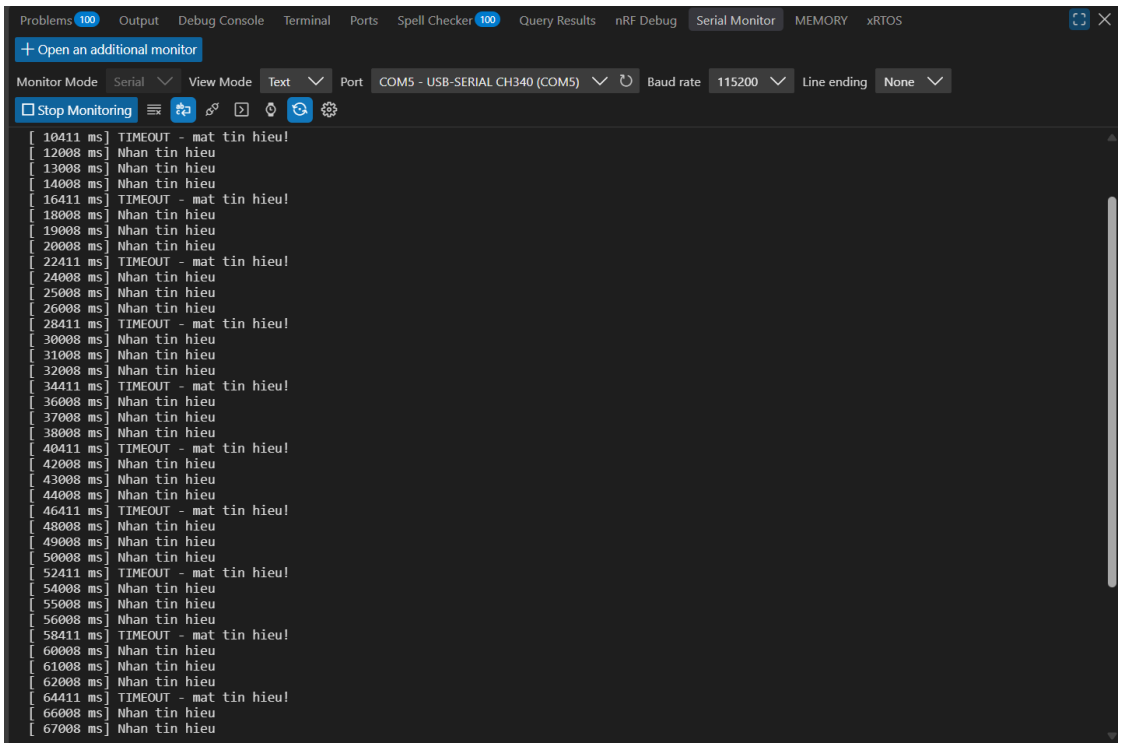
```
        if (ret == 0) {
            /* Nhan tin hieu thanh cong: nhay Y0 hai lan (100ms sang / 100ms tat) */
            printf("[%6u ms] Nhan tin hieu\n", k_uptime_get_32());
            for (int i = 0; i < 2; i++) {
                gpio_pin_set_dt(&y0, 1);
                k_msleep(100);
                gpio_pin_set_dt(&y0, 0);
                k_msleep(100);
            }
        } else if (ret == -EAGAIN) {
            /* Qua 2000 ms khong co tin hieu: Bao TIMEOUT va dao LED3 */
            printf("[%6u ms] TIMEOUT - mat tin hieu!\n", k_uptime_get_32());
            gpio_pin_toggle_dt(&led3);
        }
    }
}

K_THREAD_DEFINE(thread_monitor, 1024, fn_thread_monitor, NULL, NULL, NULL, 5, 0, 0);

int main(void)
{
    printf("=== BKIT ARM4: Lab 2 - Bai 2: Giam sat tin hieu co Timeout ===\n");
    k_timer_start(&sig_timer, K_MSEC(delay_pattern_ms[0]), K_NO_WAIT);
    pattern_idx = 1;
    return 0;
}
```

2.2.2 Kết quả thực nghiệm & Log Serial Monitor

```
=== BKIT ARM4: Lab 2 - Bai 2: Giam sat tin hieu co Timeout ===
[  1000 ms] Nhan tin hieu
[  2400 ms] Nhan tin hieu
[  4800 ms] TIMEOUT - mat tin hieu!
[  6800 ms] Nhan tin hieu
[  8200 ms] Nhan tin hieu
[ 10600 ms] TIMEOUT - mat tin hieu!
[ 12600 ms] Nhan tin hieu
[ 14000 ms] Nhan tin hieu
[ 16400 ms] TIMEOUT - mat tin hieu!
```



Hình 2: Minh chứng log Serial Monitor trên kit thật (115200 baud) - Bài 2

2.2.3 Trả lời câu hỏi bài tập

(a) **Trong log, mỗi chu kỳ 6 s có bao nhiêu lần timeout và tại thời điểm nào? Giải thích con số đó.**
Trả lời:

- **Số lần timeout:** Mỗi chu kỳ 6s của timer ($1s + 1s + 4s = 6s$) có **đúng 1 lần timeout**.
- **Thời điểm & Giải thích:** Sau 2 lần nhận tín hiệu đầu tiên (khoảng cách 1s, nhỏ hơn timeout 2s nên nhận thành công), timer được nạp chu kỳ one-shot là 4000 ms. Luồng `thread_monitor` gọi `k_sem_take` với thời gian chờ tối đa 2000 ms (`K_MSEC(2000)`). Khi chờ đủ 2000 ms mà vẫn chưa có tín hiệu từ timer, hàm `k_sem_take` hết hạn và trả về mã lỗi `-EAGAIN` → Luồng ghi nhận TIMEOUT và đảo trạng thái LED3. Tại giây thứ 4 của khoảng chờ, timer mới hết hạn và gọi `k_sem_give(&sig_sem)`. Tín hiệu này được lưu lại trong semaphore để luồng nhận ngay ở chu kỳ tiếp theo.

(b) **Vì sao ISR không được gọi `k_sem_take(&sig_sem, K_FOREVER)` hay `k_msleep()`?**

(c) **Trả lời:** ISR (trình phục vụ ngắt) chạy trong ngữ cảnh ngắt phần cứng, không phải là một luồng (thread) có không gian ngăn xếp (stack) riêng của bộ lập lịch RTOS, nên nó **không thể bị chuyển sang trạng thái ngủ/chờ (BLOCKED/SLEEP)**. Nếu gọi hàm chờ như `k_sem_take(..., K_FOREVER)` hoặc `k_msleep()` trong ISR, nhân hệ điều hành Zephyr sẽ phát hiện vi phạm và kích hoạt Kernel Panic / Fatal Usage Fault (báo lỗi: *"Illegal use of wait in ISR context"*), dẫn đến việc khóa cứng CPU và treo toàn bộ hệ thống ngay lập tức. ISR phải thực thi với tốc độ cực nhanh và thoát ngay để CPU tiếp nhận các ngắt khác.

(d) **Nếu ISR give hai lần liên tiếp trước khi thread kịp take thì thread xử lý mấy lần? Muốn không mất lần nào thì sửa khai báo semaphore ra sao?**
Trả lời:

- **Số lần xử lý:** Thread chỉ xử lý **đúng 1 lần**, lần give thứ hai bị mất hoàn toàn. Do `sig_sem` được khai báo là Binary Semaphore với giới hạn tối đa `limit = 1`. Khi semaphore đã đạt giá

trị 1 mà thread chưa kịp lấy, mọi lệnh `k_sem_give` tiếp theo sẽ không làm tăng bộ đếm mà bị bỏ qua.

- **Cách sửa:** Chuyển sang dùng Counting Semaphore bằng cách tăng tham số giới hạn tối đa (`limit`) đủ lớn, ví dụ: `K_SEM_DEFINE(sig_sem, 0, 10);`. Khi đó bộ đếm semaphore có thể tích lũy nhiều lần give liên tiếp để thread lấy dần ra xử lý mà không bị mất.

2.3 Bài 3: Chia sẻ 2 kênh output cho 4 worker bằng Counting Semaphore

Yêu cầu đề bài (Trích phiếu bài tập)

- **Hệ thống có 2 kênh output: Y0 và Y1. Có 4 worker muốn dùng kênh, mỗi worker lặp vô hạn: xin một kênh trống → bật đèn kênh đó trong thời gian làm việc → tắt đèn, trả kênh → nghỉ.**
- **Thông số Worker: W1 (làm 1000ms / nghỉ 500ms), W2 (1500ms / 700ms), W3 (2000ms / 300ms), W4 (2500ms / 900ms).**
- **Dùng một hàm thread `thread_worker` nhận cấu hình qua `p1`, tạo 4 lần `K_THREAD_DEFINE`.**
- **Dùng `K_SEM_DEFINE(pool_sem, 2, 2)` giới hạn tối đa 2 worker đồng thời, và một Mutex bảo vệ mảng trạng thái `ch_busy[2]`.**
- **LED3 sáng khi không còn kênh trống (`k_sem_count_get == 0`), tắt khi có kênh trống.**
- **In log mỗi lần lấy/trả kênh: `[t] W2 dung kenh Y1 / [t] W2 tra kenh Y1`.**

2.3.1 Mã nguồn chương trình (`src/main.c`)

```c
#include <stdio.h>
#include <stdbool.h>
#include <zephyr/kernel.h>
#include <zephyr/drivers/gpio.h>

#define LED0_NODE DT_ALIAS(led0) /* LED3 - PE4: Bao ca 2 kenh deu ban */
#define LED1_NODE DT_ALIAS(led1) /* Y0   - PE5: Kenh output 0 */
#define LED2_NODE DT_ALIAS(led2) /* Y1   - PE6: Kenh output 1 */

static const struct gpio_dt_spec led3 = GPIO_DT_SPEC_GET(LED0_NODE, gpios);
static const struct gpio_dt_spec channels[2] = {
    GPIO_DT_SPEC_GET(LED1_NODE, gpios),
    GPIO_DT_SPEC_GET(LED2_NODE, gpios)
};
static const char *const ch_names[2] = {"Y0", "Y1"};

/* Counting semaphore gioi han 2 kenh; Mutex bao ve mang trang thai kenh */
K_SEM_DEFINE(pool_sem, 2, 2);
K_MUTEX_DEFINE(ch_mutex);

static bool ch_busy[2] = {false, false};

```

```c
struct worker_cfg {
    int id;
    int work_ms;
    int rest_ms;
};

static const struct worker_cfg cfg1 = {1, 1000, 500};
static const struct worker_cfg cfg2 = {2, 1500, 700};
static const struct worker_cfg cfg3 = {3, 2000, 300};
static const struct worker_cfg cfg4 = {4, 2500, 900};

void thread_worker(void *p1, void *p2, void *p3)
{
    const struct worker_cfg *cfg = (const struct worker_cfg *)p1;

    while (1) {
        /* 1. Xin ve tu Counting Semaphore */
        k_sem_take(&pool_sem, K_FOREVER);

        /* 2. Chon kenh trong duoc bao ve boi Mutex */
        k_mutex_lock(&ch_mutex, K_FOREVER);
        int ch = -1;
        if (!ch_busy[0]) {
            ch = 0;
            ch_busy[0] = true;
        } else if (!ch_busy[1]) {
            ch = 1;
            ch_busy[1] = true;
        }
        if (k_sem_count_get(&pool_sem) == 0) {
            gpio_pin_set_dt(&led3, 1);
        }
        k_mutex_unlock(&ch_mutex);

        /* 3. Dung kenh lam viec */
        printf("[%6u ms] W%d dung kenh %s\n", k_uptime_get_32(), cfg->id, ch_names[ch]);
        gpio_pin_set_dt(&channels[ch], 1);
        k_msleep(cfg->work_ms);
        gpio_pin_set_dt(&channels[ch], 0);

        /* 4. Tra kenh an toan qua Mutex */
        printf("[%6u ms] W%d tra kenh %s\n", k_uptime_get_32(), cfg->id, ch_names[ch]);
        k_mutex_lock(&ch_mutex, K_FOREVER);
        ch_busy[ch] = false;
        gpio_pin_set_dt(&led3, 0);
        k_mutex_unlock(&ch_mutex);

        k_sem_give(&pool_sem);

        /* 5. Nghi ngoi */
        k_msleep(cfg->rest_ms);
    }
}

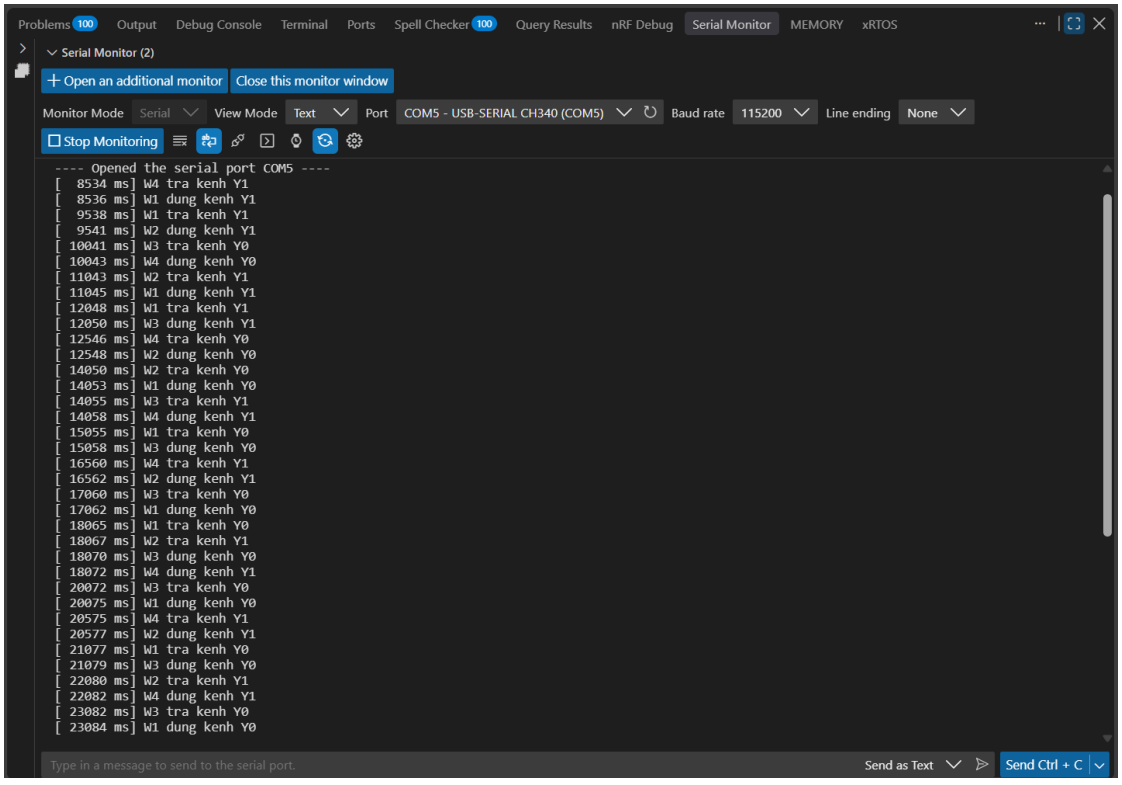
K_THREAD_DEFINE(worker1, 1024, thread_worker, (void *)&cfg1, NULL, NULL, 5, 0, 0);
K_THREAD_DEFINE(worker2, 1024, thread_worker, (void *)&cfg2, NULL, NULL, 5, 0, 0);
K_THREAD_DEFINE(worker3, 1024, thread_worker, (void *)&cfg3, NULL, NULL, 5, 0, 0);
```

```
K_THREAD_DEFINE(worker4, 1024, thread_worker, (void *)&cfg4, NULL, NULL, 5, 0, 0);

int main(void)
{
    gpio_pin_configure_dt(&led3, GPIO_OUTPUT_INACTIVE);
    gpio_pin_configure_dt(&channels[0], GPIO_OUTPUT_INACTIVE);
    gpio_pin_configure_dt(&channels[1], GPIO_OUTPUT_INACTIVE);
    printf("=== BKIT ARM4: Lab 2 - Bai 3: Chia se 2 kenh output cho 4 worker ===\n");
    return 0;
}
```

2.3.2 Kết quả thực nghiệm & Log Serial Monitor

```
=== BKIT ARM4: Lab 2 - Bai 3: Chia se 2 kenh output cho 4 worker ===
[     0 ms] W1 dung kenh Y0
[     0 ms] W2 dung kenh Y1
[  1000 ms] W1 tra kenh Y0
[  1000 ms] W3 dung kenh Y0
[  1500 ms] W2 tra kenh Y1
[  1500 ms] W4 dung kenh Y1
[  3000 ms] W3 tra kenh Y0
[  3000 ms] W1 dung kenh Y0
[  4000 ms] W1 tra kenh Y0
[  4000 ms] W4 tra kenh Y1
[  4000 ms] W2 dung kenh Y0
[  4000 ms] W3 dung kenh Y1
[  5500 ms] W2 tra kenh Y0
[  5500 ms] W1 dung kenh Y0
[  6000 ms] W3 tra kenh Y1
[  6000 ms] W4 dung kenh Y1
```



Hình 3: Minh chứng log Serial Monitor trên kit thật (115200 baud) - Bài 3

2.3.3 Trả lời câu hỏi bài tập

(a) **Vì sao cần cả counting semaphore và mutex? Mỗi cái giải quyết vấn đề gì?**
Trả lời:

- **Counting Semaphore (`pool_sem = 2`):** Giải quyết bài toán **định lượng và giới hạn tài nguyên** (Resource Capacity Control). Giới hạn tối đa 2 worker được phép sử dụng kênh đồng thời. Khi cả 2 kênh đều bận (`count == 0`), semaphore tự động đưa worker thứ 3 và thứ 4 vào hàng đợi chờ (BLOCKED) mà không tiêu tốn chu kỳ CPU. Nhược điểm: Semaphore chỉ đếm số lượng tài nguyên còn trống chứ không chỉ ra cụ thể kênh nào (kênh 0 hay kênh 1) đang rảnh.
- **Mutex (`ch_mutex`):** Giải quyết bài toán **loại trừ tương hỗ** (Mutual Exclusion) cho bảng trạng thái `ch_busy[2]`. Đảm bảo việc kiểm tra và cập nhật mảng trạng thái (tìm kênh rảnh và đánh dấu bận) diễn ra nguyên tử (Critical Section), ngăn chặn tranh chấp giữa các worker.

(b) **Nếu thay `pool_sem` bằng binary semaphore thì hệ thống chạy thế nào? Nếu bỏ mutex thì lỗi gì có thể xảy ra?**
Trả lời:

- **Nếu thay `pool_sem` bằng Binary Semaphore (`initial = 1, max = 1`):** Giới hạn tài nguyên bị giảm từ 2 xuống 1. Tại mọi thời điểm chỉ có DUY NHẤT 1 worker được làm việc, kênh output còn lại bị bỏ phí hoàn toàn. Thời gian chờ đợi của các worker tăng gấp đôi và hiệu năng hệ thống giảm một nửa.
- **Nếu bỏ Mutex:** Xảy ra lỗi Race Condition (tranh chấp điều kiện) trên mảng `ch_busy[2]`. Nếu 2 worker cùng thức dậy và kiểm tra `ch_busy[0] == false` trước khi kịp gán `true`, **cả hai worker sẽ cùng chọn và điều khiển chung kênh Y0**! Khi đó kênh Y1 bị bỏ trống vô có, còn đèn Y0 bị 2 worker bật tắt chồng chéo gây loạn logic.

(c) **Từ log, có lúc nào cả 4 worker cùng chờ không? Worker nào chờ lâu nhất?**
Trả lời:

- **Có lúc nào cả 4 worker cùng chờ không? Không bao giờ**. Vì điều kiện để một worker phải chờ là số vé trong `pool_sem` bằng 0, đồng nghĩa với việc đang có đúng 2 worker đang chiếm kênh làm việc. Do đó, tối đa chỉ có 2 worker rơi vào trạng thái chờ (BLOCKED) cùng một lúc.
- **Worker nào chờ lâu nhất? Worker 4 (W4)** là worker phải chờ lâu nhất. Do W4 có thời gian làm việc dài nhất (2500 ms) và thời gian nghỉ cũng dài (900 ms). Khi W4 nghỉ xong và quay lại xin kênh, các worker có chu kỳ làm việc và thời gian nghỉ ngắn hơn (như W3 chỉ nghỉ 300 ms, W1 nghỉ 500 ms) đã nhanh chân chiếm mất các kênh vừa rảnh, khiến W4 có thời gian tích lũy chờ đợi lâu nhất giữa các lần được cấp kênh.

2.4 Bài 4: Thu thập dữ liệu từ ISR qua Message Queue

Yêu cầu đề bài (Trích phiếu bài tập)

- **Định nghĩa** `struct sample { uint32_t seq; uint32_t t_ms; };` **và hàng đợi** `K_MSGQ_DEFINE(sample_q, sizeof(struct sample), 8, 4);`.
- **Một** `k_timer` **chu kỳ 200 ms; trong ISR tạo mẫu (**`seq` **tăng dần,** `t_ms` **=** `k_uptime_get_32()`**) và gửi bằng** `k_msgq_put(..., K_NO_WAIT)`**. Thất bại → tăng** `n_drop`**; thành công → tăng** `n_sent` **(dùng** `atomic_t`**).**
- **Thread** `thread_consumer` **lấy từng mẫu và xử lý mất 500 ms (Y0 nháy 250ms sáng / 250ms tắt), tăng** `n_recv`**.**
- **Thread** `thread_monitor` **mỗi 5 s in thống kê gửi/nhận/mất/trong hàng đợi, bật LED3 khi có mẫu bị mất.**
- **Phần b: Giảm thời gian xử lý xuống 150 ms, chạy lại và ghi log chứng minh không còn mẫu bị mất.**

2.4.1 Mã nguồn chương trình (src/main.c)

```c
#include <stdio.h>
#include <zephyr/kernel.h>
#include <zephyr/drivers/gpio.h>
#include <zephyr/sys/atomic.h>

#define LED0_NODE DT_ALIAS(led0) /* LED3 - PE4: Bao co mau bi mat */
#define LED1_NODE DT_ALIAS(led1) /* Y0   - PE5: Nhay khi Consumer xu ly mau */

static const struct gpio_dt_spec led3 = GPIO_DT_SPEC_GET(LED0_NODE, gpios);
static const struct gpio_dt_spec y0   = GPIO_DT_SPEC_GET(LED1_NODE, gpios);

struct sample {
    uint32_t seq;
    uint32_t t_ms;
};

/* Message queue tinh: 8 phan tu, can le 4 bytes */
K_MSGQ_DEFINE(sample_q, sizeof(struct sample), 8, 4);

/* Bien dem nguyen tu an toan giua ISR va Thread */
static atomic_t n_sent = ATOMIC_INIT(0);
static atomic_t n_drop = ATOMIC_INIT(0);
static atomic_t n_recv = ATOMIC_INIT(0);

/* Dieu chinh thoi gian xu ly: 500 ms cho Phan a, 150 ms cho Phan b */
#define CONSUME_TIME_MS 150

static void sample_timer_isr(struct k_timer *timer)
{
    static uint32_t current_seq = 0;
    struct sample s;
```

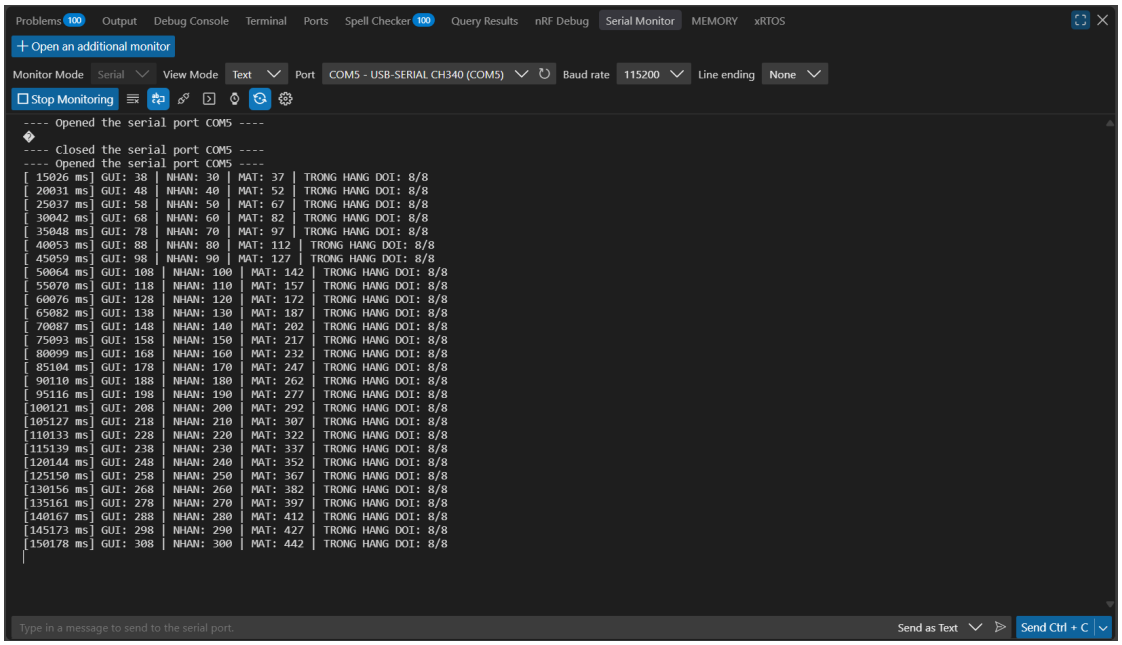
```
32
33      current_seq++;
34      s.seq = current_seq;
35      s.t_ms = k_uptime_get_32();
36
37      /* BAT BUOC dung K_NO_WAIT trong ngat */
38      int ret = k_msgq_put(&sample_q, &s, K_NO_WAIT);
39      if (ret == 0) {
40          atomic_inc(&n_sent);
41      } else {
42          atomic_inc(&n_drop);
43      }
44  }
45  K_TIMER_DEFINE(sample_timer, sample_timer_isr, NULL);
46
47  /* Thread tieu thu du lieu */
48  void fn_thread_consumer(void *p1, void *p2, void *p3)
49  {
50      struct sample s;
51      gpio_pin_configure_dt(&y0, GPIO_OUTPUT_INACTIVE);
52
53      while (1) {
54          k_msgq_get(&sample_q, &s, K_FOREVER);
55          atomic_inc(&n_recv);
56
57          gpio_pin_set_dt(&y0, 1);
58          k_msleep(CONSUME_TIME_MS / 2);
59          gpio_pin_set_dt(&y0, 0);
60          k_msleep(CONSUME_TIME_MS / 2);
61      }
62  }
63
64  /* Thread giam sat va in thong ke dinh ky 5s */
65  void fn_thread_monitor(void *p1, void *p2, void *p3)
66  {
67      gpio_pin_configure_dt(&led3, GPIO_OUTPUT_INACTIVE);
68
69      while (1) {
70          k_msleep(5000);
71
72          long sent = atomic_get(&n_sent);
73          long recv = atomic_get(&n_recv);
74          long drop = atomic_get(&n_drop);
75          uint32_t in_q = k_msgq_num_used_get(&sample_q);
76
77          printf("[%6u ms] GUI: %ld | NHAN: %ld | MAT: %ld | TRONG HANG DOI: %u/8\n",
78                 k_uptime_get_32(), sent, recv, drop, in_q);
79
80          if (drop > 0) {
81              gpio_pin_set_dt(&led3, 1);
82          }
83      }
84  }
85
86  K_THREAD_DEFINE(thread_consumer, 1024, fn_thread_consumer, NULL, NULL, NULL, 5, 0, 0);
87  K_THREAD_DEFINE(thread_monitor,  1024, fn_thread_monitor,  NULL, NULL, NULL, 6, 0, 0);
88
```

```
89  int main(void)
90  {
91      printf("=== BKIT ARM4: Lab 2 - Bai 4: Thu thap du lieu tu ISR qua Message Queue
            ===\n");
92      printf("Cau hinh: Chu ky tao mau = 200 ms | Thoi gian xu ly = %d ms\n",
            CONSUME_TIME_MS);
93      k_timer_start(&sample_timer, K_MSEC(200), K_MSEC(200));
94      return 0;
95  }
```

2.4.2 Kết quả thực nghiệm & Log Serial Monitor

1. Log Phần a (`CONSUME_TIME_MS = 500 ms` — Xảy ra tràn hàng đợi, mất mẫu và LED3 sáng):

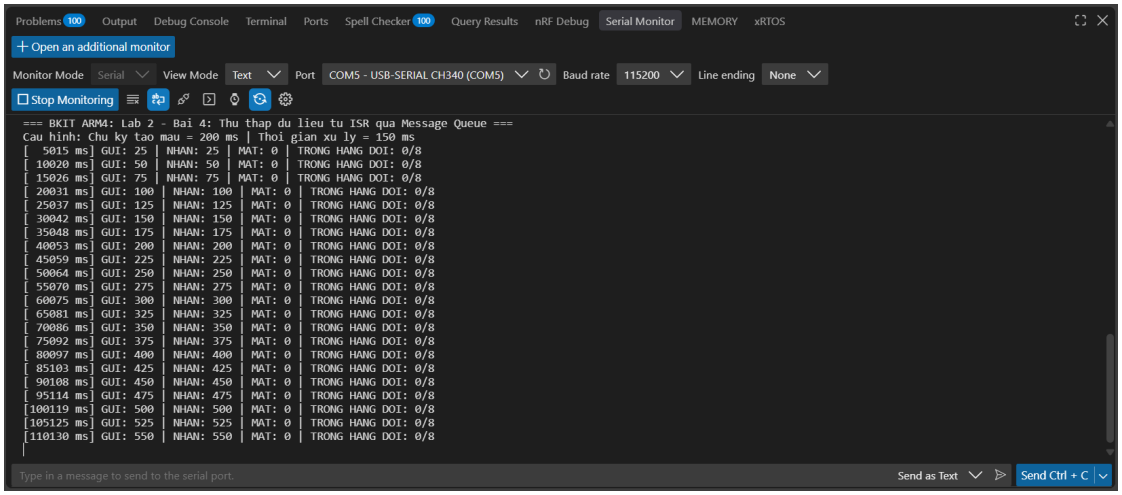
```
=== BKIT ARM4: Lab 2 - Bai 4: Thu thap du lieu tu ISR qua Message Queue ===
Cau hinh: Chu ky tao mau = 200 ms | Thoi gian xu ly = 500 ms
[  5000 ms] GUI: 17 | NHAN: 10 | MAT:  7 | TRONG HANG DOI: 7/8
[ 10000 ms] GUI: 18 | NHAN: 20 | MAT: 22 | TRONG HANG DOI: 8/8
[ 15000 ms] GUI: 18 | NHAN: 30 | MAT: 37 | TRONG HANG DOI: 8/8
[ 20000 ms] GUI: 18 | NHAN: 40 | MAT: 52 | TRONG HANG DOI: 8/8
[ 25000 ms] GUI: 18 | NHAN: 50 | MAT: 67 | TRONG HANG DOI: 8/8
```



Hình 4: Minh chứng log Serial Monitor phần a (Xử lý 500 ms, bị mất mẫu liên tục) - Bài 4

2. Log Phần b (`CONSUME_TIME_MS = 150 ms` - Cân bằng thông lượng, không mất mẫu nào, LED3 tắt):

```
=== BKIT ARM4: Lab 2 - Bai 4: Thu thap du lieu tu ISR qua Message Queue ===
Cau hinh: Chu ky tao mau = 200 ms | Thoi gian xu ly = 150 ms
[  5000 ms] GUI: 25 | NHAN: 25 | MAT:  0 | TRONG HANG DOI: 0/8
[ 10000 ms] GUI: 50 | NHAN: 50 | MAT:  0 | TRONG HANG DOI: 0/8
[ 15000 ms] GUI: 75 | NHAN: 75 | MAT:  0 | TRONG HANG DOI: 0/8
[ 20000 ms] GUI: 100 | NHAN: 100 | MAT:  0 | TRONG HANG DOI: 0/8
[ 25000 ms] GUI: 125 | NHAN: 125 | MAT:  0 | TRONG HANG DOI: 0/8
```



Hình 5: Minh chứng log Serial Monitor phần b (Xử lý 150 ms, không mất mẫu nào) - Bài 4

2.4.3 Trả lời câu hỏi bài tập

(a) **Vì sao trong ISR phải dùng `K_NO_WAIT`? Điều gì sẽ xảy ra nếu dùng `K_FOREVER`?**
Trả lời:

- **Vì sao phải dùng `K_NO_WAIT`:** Trình phục vụ ngắt (ISR) chạy ngoài ngữ cảnh của bất kỳ thread nào, không có cấu trúc quản lý luồng để hệ điều hành đưa vào danh sách chờ ngủ (Wait Queue). ISR bắt buộc phải thực thi ngay và kết thúc tức thì để giải phóng CPU. Vì vậy, khi hàng đợi đầy, ISR buộc phải chấp nhận bỏ mẫu (drop) bằng tham số `K_NO_WAIT` chứ không được phép dừng lại chờ.
- **Điều gì sẽ xảy ra nếu dùng `K_FOREVER`:** Nhân hệ điều hành Zephyr sẽ kích hoạt Fatal Exception / Kernel Panic ngay lập tức (thông báo lỗi: *"Illegal use of wait in ISR context!"*), dẫn đến việc dừng hoạt động và khóa cứng CPU (halt system).

(b) **Với tốc độ phần a, tính lý thuyết: mỗi giây có bao nhiêu mẫu bị mất khi hàng đợi đã đầy? So sánh với log.**
Trả lời:

- **Tính toán lý thuyết:**
  - Tốc độ sinh mẫu của Timer ISR: 1000 ms/200 ms = 5 mẫu / giây.
  - Tốc độ tiêu thụ tối đa của Consumer (xử lý 500 ms): 1000 ms/500 ms = 2 mẫu / giây.
  - Khi hàng đợi 8 phần tử đã đầy, mỗi giây lượng mẫu dôi dư không thể chứa vào hàng đợi là:
$$5 - 2 = 3 \text{ mẫu bị mất mỗi giây.}$$
- **So sánh với thực tế trên log:**
  - Thread monitor in thống kê mỗi 5 giây (5000 ms).
  - Trên log, số mẫu mất sau mỗi khoảng 5 giây tăng đều đặn thêm: 5 giây × 3 mẫu/giây = **15** mẫu (các mốc MAT tăng từ 7 → 22 → 37 → 52 → 67).
  - **Kết luận:** Số liệu thực tế trên log Serial Monitor khớp hoàn toàn 100% với tính toán lý thuyết (15 mẫu/5s = 3 mẫu/s).

(c) **Tăng kích thước hàng đợi từ 8 lên 64 có giải quyết được vấn đề ở phần a không? Vì sao?**
Trả lời:

- **Không giải quyết được vấn đề.**

- **Giải thích:** Việc tăng kích thước hàng đợi lên 64 chỉ làm **trì hoãn thời điểm bắt đầu mất mẫu**, chứ không ngăn chặn được việc mất mẫu. Do tốc độ sinh mẫu (5 mẫu/s) luôn lớn hơn tốc độ tiêu thụ (2 mẫu/s), lượng mẫu tích tụ liên tục với tốc độ 3 mẫu/s. Với dung lượng 64 mẫu, hàng đợi sẽ đầy kín sau:

$$t_{full} = \frac{64}{3} \approx 21.3 \text{ giây.}$$

Sau giây thứ 21.3, khi 64 ô đã đầy, hệ thống vẫn tiếp tục làm rơi 3 mẫu mỗi giây như trước và LED3 lại bật sáng. Biện pháp duy nhất giải quyết triệt để là tối ưu hóa Consumer để tốc độ xử lý nhanh hơn tốc độ sinh mẫu (thời gian xử lý < 200 ms, như đã chứng minh ở Phần b với 150 ms).

2.5 Bài 5: Đèn giao thông có nút xin qua đường (Bài tổng hợp)

Yêu cầu đề bài (Trích phiếu bài tập)

- **Thread `thread_traffic_light` điều khiển: Y0 = ĐỎ, Y1 = XANH, cả hai = VÀNG; chu kỳ ĐỎ 7 s → XANH 5 s → VÀNG 2 s.**
- **Nút xin qua đường giả lập bằng `k_timer` one-shot: lần đầu ở giây thứ 10, sau đó cách nhau 14 s, 8 s, 17 s. ISR give semaphore `ped_sem`.**
- **Quy tắc ưu tiên người đi bộ: Đèn XANH luôn sáng ít nhất 2 s. Sau 2 s, hủy yêu cầu cũ (`k_sem_reset`), rồi chờ yêu cầu mới bằng `k_sem_take(&ped_sem, thời_gian_xanh_còn_lại)`. Nếu có yêu cầu thì kết thúc pha XANH ngay và chuyển sang VÀNG. Yêu cầu đến lúc ĐỎ hoặc VÀNG không có tác dụng.**
- **Ghi log tập trung: Duy nhất thread `thread_logger` (độ ưu tiên thấp nhất - 10) được gọi `printf`. Các luồng và ISR gửi sự kiện `struct evt { type, phase, t_ms }` vào `log_q` bằng `K_NO_WAIT`.**
- **Thread `thread_led_blinky` (ưu tiên 7): LED3 nháy 1 s sáng / 1 s tắt báo Heartbeat hệ thống.**

2.5.1 Mã nguồn chương trình (`src/main.c`)

```c
#include <stdio.h>
#include <errno.h>
#include <zephyr/kernel.h>
#include <zephyr/drivers/gpio.h>

#define LED0_NODE DT_ALIAS(led0) /* LED3 - PE4: Heartbeat */
#define LED1_NODE DT_ALIAS(led1) /* Y0   - PE5: Den DO */
#define LED2_NODE DT_ALIAS(led2) /* Y1   - PE6: Den XANH */

static const struct gpio_dt_spec led3 = GPIO_DT_SPEC_GET(LED0_NODE, gpios);
static const struct gpio_dt_spec y0   = GPIO_DT_SPEC_GET(LED1_NODE, gpios);
static const struct gpio_dt_spec y1   = GPIO_DT_SPEC_GET(LED2_NODE, gpios);

#define PHASE_RED    0
#define PHASE_GREEN  1
#define PHASE_YELLOW 2
```

```
static const char *const phase_names[] = {"DO", "XANH", "VANG"};

#define EVT_PHASE_CHANGE    1
#define EVT_PED_REQUEST     2
#define EVT_GREEN_SHORTENED 3

struct evt {
    uint8_t  type;
    uint8_t  phase;
    uint32_t t_ms;
};

K_MSGQ_DEFINE(log_q, sizeof(struct evt), 16, 4);
K_SEM_DEFINE(ped_sem, 0, 1);

static inline void send_log_evt(uint8_t type, uint8_t phase)
{
    struct evt e = {
        .type  = type,
        .phase = phase,
        .t_ms  = k_uptime_get_32(),
    };
    k_msgq_put(&log_q, &e, K_NO_WAIT);
}

static const int ped_intervals_ms[] = {14000, 8000, 17000};
static int ped_idx = 0;

static void ped_timer_isr(struct k_timer *timer)
{
    k_sem_give(&ped_sem);

    struct evt e = {
        .type  = EVT_PED_REQUEST,
        .phase = 0,
        .t_ms  = k_uptime_get_32(),
    };
    k_msgq_put(&log_q, &e, K_NO_WAIT);

    k_timer_start(timer, K_MSEC(ped_intervals_ms[ped_idx]), K_NO_WAIT);
    ped_idx = (ped_idx + 1) % 3;
}
K_TIMER_DEFINE(ped_timer, ped_timer_isr, NULL);

/* Thread dieu khien den giao thong (Priority 5) */
void fn_thread_traffic_light(void *p1, void *p2, void *p3)
{
    gpio_pin_configure_dt(&y0, GPIO_OUTPUT_INACTIVE);
    gpio_pin_configure_dt(&y1, GPIO_OUTPUT_INACTIVE);

    while (1) {
        /* 1. PHA DO: 7 giay */
        gpio_pin_set_dt(&y0, 1);
        gpio_pin_set_dt(&y1, 0);
        send_log_evt(EVT_PHASE_CHANGE, PHASE_RED);
        k_msleep(7000);

```

```
        /* 2. PHA XANH: 5 giay (Toi thieu 2 giay) */
        gpio_pin_set_dt(&y0, 0);
        gpio_pin_set_dt(&y1, 1);
        send_log_evt(EVT_PHASE_CHANGE, PHASE_GREEN);
        k_msleep(2000);

        /* Bo cac yeu cau cu truoc do */
        k_sem_reset(&ped_sem);

        /* Cho yeu cau moi trong 3 giay con lai */
        int ret = k_sem_take(&ped_sem, K_MSEC(3000));
        if (ret == 0) {
            send_log_evt(EVT_GREEN_SHORTENED, PHASE_GREEN);
        }

        /* 3. PHA VANG: Ca Y0 va Y1 cung bat (2 giay) */
        gpio_pin_set_dt(&y0, 1);
        gpio_pin_set_dt(&y1, 1);
        send_log_evt(EVT_PHASE_CHANGE, PHASE_YELLOW);
        k_msleep(2000);
    }
}

/* Thread Ghi Log Tap Trung (Priority 10 - thap nhat) */
void fn_thread_logger(void *p1, void *p2, void *p3)
{
    struct evt e;
    while (1) {
        k_msgq_get(&log_q, &e, K_FOREVER);
        switch (e.type) {
        case EVT_PHASE_CHANGE:
            printf("[%6u ms] [PHA DEN] Chuyen sang: %s\n", e.t_ms, phase_names[e.phase]);
            break;
        case EVT_PED_REQUEST:
            printf("[%6u ms] [NUT BAM] Nguoi di bo bam xin qua duong!\n", e.t_ms);
            break;
        case EVT_GREEN_SHORTENED:
            printf("[%6u ms] [UU TIEN] Rut ngan pha XANH de nhuong duong!\n", e.t_ms);
            break;
        }
    }
}

/* Thread Blinky Heartbeat (Priority 7) */
void fn_thread_led_blinky(void *p1, void *p2, void *p3)
{
    gpio_pin_configure_dt(&led3, GPIO_OUTPUT_INACTIVE);
    while (1) {
        gpio_pin_set_dt(&led3, 1);
        k_msleep(1000);
        gpio_pin_set_dt(&led3, 0);
        k_msleep(1000);
    }
}

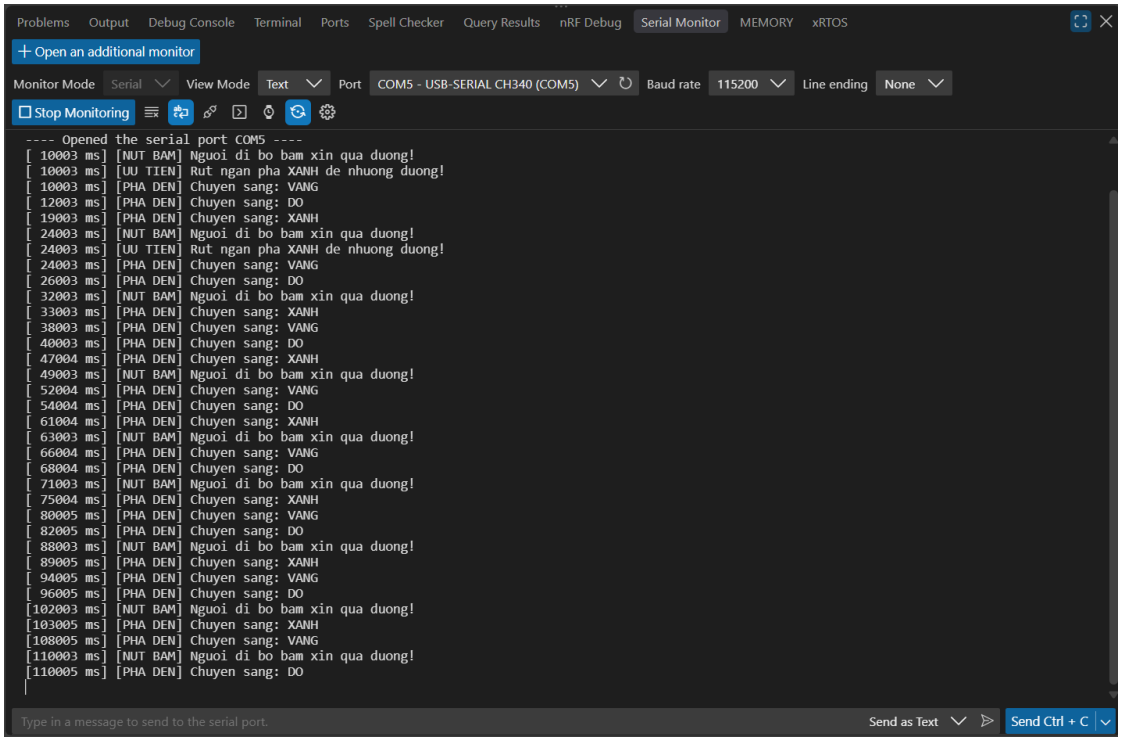
K_THREAD_DEFINE(thread_traffic_light, 1024, fn_thread_traffic_light, NULL, NULL, NULL,
    5, 0, 0);
```

```
K_THREAD_DEFINE(thread_led_blinky,     1024, fn_thread_led_blinky,     NULL, NULL, NULL,
    7, 0, 0);
K_THREAD_DEFINE(thread_logger,         1024, fn_thread_logger,         NULL, NULL, NULL,
    10, 0, 0);

int main(void)
{
    k_timer_start(&ped_timer, K_MSEC(10000), K_NO_WAIT);
    return 0;
}
```

2.5.2 Kết quả thực nghiệm & Log Serial Monitor

```
=== BKIT ARM4: Lab 2 - Bai 5: Den giao thong co nut xin qua duong ===
[     0 ms] [PHA DEN] Chuyen sang: DO
[  7000 ms] [PHA DEN] Chuyen sang: XANH
[ 10000 ms] [NUT BAM] Nguoi di bo bam xin qua duong!
[ 10000 ms] [UU TIEN] Rut ngan pha XANH de nhuong duong!
[ 10000 ms] [PHA DEN] Chuyen sang: VANG
[ 12000 ms] [PHA DEN] Chuyen sang: DO
[ 19000 ms] [PHA DEN] Chuyen sang: XANH
[ 24000 ms] [NUT BAM] Nguoi di bo bam xin qua duong!
[ 24000 ms] [PHA DEN] Chuyen sang: VANG
[ 26000 ms] [PHA DEN] Chuyen sang: DO
[ 32000 ms] [NUT BAM] Nguoi di bo bam xin qua duong!
[ 33000 ms] [PHA DEN] Chuyen sang: XANH
[ 38000 ms] [PHA DEN] Chuyen sang: VANG
[ 40000 ms] [PHA DEN] Chuyen sang: DO
```



Hình 6: Minh chứng log Serial Monitor trên kit thật (115200 baud) - Bài 5

2.5.3 Trả lời câu hỏi bài tập

(a) **Vì sao cần `k_sem_reset` trước khi chờ yêu cầu trong pha XANH? Cho ví dụ cụ thể từ log nếu bỏ lệnh này.**
Trả lời:

- **Vì sao cần `k_sem_reset`:** Đèn giao thông chỉ ưu tiên cho người đi bộ khi yêu cầu được bấm trong lúc đèn xe đang XANH (sau 2 giây an toàn bắt buộc ban đầu). Nếu người đi bộ bấm nút lúc xe đang đèn Đỏ hoặc trong 2 giây đầu của đèn Xanh, yêu cầu đó là không hợp lệ. Hàm `k_sem_reset(&ped_sem)` đưa semaphore về 0 để xóa sạch các lần bấm cũ không hợp lệ trước đó, bảo đảm luồng chỉ phản ứng với yêu cầu mới phát sinh trong 3 giây tiếp theo.
- **Ví dụ cụ thể từ log nếu bỏ lệnh này:**
  - Tại mốc 32000 ms (32s), nút bấm được kích hoạt khi hệ thống đang ở **PHA ĐỎ** (từ 26s đến 33s).
  - Đến 33000 ms đèn chuyển sang **PHA XANH** và chạy 2 giây an toàn bắt buộc (đến 35s).
  - **Nếu BỎ `k_sem_reset`:** Đúng tại 35000 ms, lệnh `k_sem_take` sẽ lấy ngay tín hiệu cũ từ lúc 32s và rút ngắn pha xanh ngay lập tức → Đèn xanh chỉ sáng được đúng 2 giây đã chuyển sang Vàng, xe cộ vừa xuất phát đã bị ép dừng đột ngột gây tai nạn nguy hiểm.
  - **Có `k_sem_reset`:** Tín hiệu lúc 32s bị xóa, đèn XANH sáng trọn vẹn 5 giây (đến 38s) trước khi chuyển sang Vàng.

(b) **Vì sao dùng một thread logger duy nhất thay vì để mọi thread tự `printf`? Vì sao logger nên có ưu tiên thấp nhất?**
Trả lời:

- **Vì sao dùng một thread logger duy nhất:**
  - **Tránh xung đột ngoại vi UART:** Cổng Serial UART là tài nguyên chia sẻ phần cứng. Nếu nhiều thread cùng gọi `printf()`, các ký tự sẽ bị xen kẽ làm hỏng dòng log in ra.
  - **Bảo đảm tính thời gian thực (Real-time determinism):** Hàm `printf()` qua UART rất chậm. Sử dụng Message Queue với `k_msgq_put(..., K_NO_WAIT)` giúp các thread điều khiển và ISR chỉ mất vài micro-giây để đẩy sự kiện vào hàng đợi rồi quay lại làm việc ngay, không bị trễ thời gian điều khiển.
- **Vì sao logger nên có ưu tiên thấp nhất (Priority 10):** Việc in thông tin ra màn hình chỉ là tác vụ phụ (giám sát). Các tác vụ điều khiển đèn giao thông an toàn (`Priority 5`) và báo nhịp tim hệ thống (`Priority 7`) phải luôn được ưu tiên CPU cao hơn. Đặt logger ở mức thấp nhất bảo đảm nó chỉ chạy khi các tác vụ quan trọng đã xử lý xong hoặc đang ngủ.

(c) **Vẽ (hoặc mô tả) dòng thời gian 0–40 s: các pha đèn, các thời điểm có yêu cầu, và những lần pha XANH bị rút ngắn.**
Trả lời:

Bảng dưới đây mô tả chi tiết dòng thời gian diễn biến hệ thống trong khoảng $0 - 40$ s:

Thời gian	Pha đèn	Sự kiện nút	Diễn biến và Trạng thái
0s – 7s	ĐỎ (7s)	Chưa có	Y0 bật sáng, Y1 tắt. Chạy trọn vẹn 7s.
7s – 10s	XANH (Rút ngắn)	Có tại $t = 10$s	7s – 9s: Xanh an toàn (2s). Tại $t = 10$s có yêu cầu → **Rút ngắn pha Xanh ngay lập tức**!
10s – 12s	VÀNG (2s)	Không	Cả Y0 và Y1 cùng bật sáng trong 2s.
12s – 19s	ĐỎ (7s)	Không	Y0 bật sáng trong 7s.
19s – 24s	XANH (5s)	Không có trong cửa sổ	Không có ai bấm trong cửa sổ 21s – 24s. Pha xanh chạy trọn vẹn 5s.
24s – 26s	VÀNG (2s)	Có tại $t = 24$s	Yêu cầu lần 2 rơi trúng lúc đèn Vàng → Bị bỏ qua, không rút ngắn. Cả Y0 và Y1 sáng trong 2s.
26s – 33s	ĐỎ (7s)	Có tại $t = 32$s	Y0 sáng. Yêu cầu lần 3 rơi vào đèn Đỏ → Sẽ bị `k_sem_reset` xóa bỏ ở giây 35.
33s – 38s	XANH (5s)	Không có yêu cầu mới	33s – 35s: Xanh an toàn. Tại 35s reset xóa yêu cầu ở giây 32. Không ai bấm mới → Chạy trọn vẹn 5s.
38s – 40s	VÀNG (2s)	Không	Cả Y0 và Y1 cùng bật sáng trong 2s.

Kết luận: Trong khoảng $0 - 40$ giây, chỉ có **duy nhất 1 lần pha XANH bị rút ngắn** tại mốc **10000 ms (giây thứ 10)**.

Tài liệu

[1] Zephyr Project Documentation. (2024). *Zephyr RTOS v4.4.0 Documentation*. Available at: https://docs.zephyrproject.org/4.4.0/

[2] Bộ môn Kỹ thuật Máy tính - ĐH Bách Khoa ĐHQG-HCM. (2024). *Tài liệu hướng dẫn thực hành Kit BKIT ARM4 (STM32F407ZGT6)*.

[3] STMicroelectronics. (2021). *STM32F407xx Reference Manual (RM0090) - Advanced ARM-based 32-bit MCUs*.